FONCTIONS D'UNE VARIABLE RÉELLE

- **Les fonctions affines.**
  - Rappeler la forme sous laquelle peut s'écrire une fonction affine.
  - f(x) = ax + b
  - Rappeler à quoi correspondent les paramètres utilisés dans cette forme.
  - a correspond au coefficient directeur
    b correspond à l'ordonnée à l'origine
  - Dans un repère orthonormé, on considère deux points A et B de coordonnées :
    A (1 ; – 3) et B (4 ; 3).
    Déterminer une équation de la fonction affine dont la courbe représentative passe par les points A et B.

    * Coeff directeur :

    a = yB - yA / xB - XA
    a = 3 -(-3) / 4-1 = 2

    * Ordonnée à l'origine
    f(x) = ax + b

    A appartient à la droite :
    -3 = 2 x 1 + b
    -3 = 2 + b
    -3 - 2 = b
    -5 = b
  - De manière générale, on considère désormais deux points A et B dans un repère orthonormé dont on note les coordonnées de la manière suivante :
    A (x_A ; y_A) et B (x_B ; y_B).
    Déterminer une équation de la fonction affine dont la courbe représentative passe par les points A et B en fonction des coordonnées des points A et B.

    * a = yB - yA / xB - XA

    * Si A appartient à la droite:
    yA = a x xA + b
    yA - a x xA = b

* D'où : f(x) = (yB - yA / xB - XA)x + yA - (yB - yA / xB - XA) x xA

- Proposer un algorithme permettant de déterminer une équation d'une fonction affine en fonction des coordonnées de deux points A et B saisies par l'utilisateur.

  Lire xA
  Lire yB
  Lire xB
  Lire yB
  a = (yB - yA) / (xB - xA)
  b = yA - a * xA
  Afficher " l'équation de la droite est y = ax + b"

- Traduire cet algorithme en langage Python.

Créer une fonction *eq_fonction_affine (xA,yA,xB,yB)* prenant en paramètres les coordonnées des points A et B et retournant l'expression de la fonction affine correspondante.

```
def eq_fonction_affine (xA, xB, yA, yB):
    coef_dir = (yA - yB) / (xA - xB)
    ordo_origine = yA - coef_dir * xA
    fonction = "y = " + str(coef_dir) + " * x + " + str(ordo_origine)
    return fonction

Xa = float(input("Entrez l'abscisse du point A "))
Ya = float(input("Entrez l'ordonnée du point A "))
Xb = float(input("Entrez l'abscisse du point B "))
Yb = float(input("Entrez l'ordonnée du point B "))
print(eq_fonction_affine(Xa, Xb, Ya, Yb))
```

- **<u>Résolution d'une équation du second degré.</u>**
  - Rappeler la forme sous laquelle peut s'écrire une équation du second degré.

    ax^¨2 + bx + c = 0

  - Rappeler les différentes solutions possibles d'une telle équation.

    discriminant(delta) = b^¨2 -4ac

    si delta = 0 : 1 seule solution
    si delta < 0 : pas de solutions
    si delta > 0 : 2 solutions

  - Proposer un algorithme permettant de déterminer les solutions d'une

équation du second degré en fonction des coefficients saisis par l'utilisateur.

```
Entrez a
Entrez b
Entrez c
delta = b**2 - 4*a*c
si delta < 0
   afficher "pas de solution"
si delta = 0
   x = -b / (2*a)
   afficher "il y a une seule solution",x
si delta > 0
    x1 = (-b - sqrt(delta) / (2*a))
    x2 = (-b + sqrt(delta) / (2*a))
   afficher "il y a deux solutions",x1,x2
```

- Traduire cet algorithme en langage Python.

```
a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))
delta = b**2 - 4*a*c
if delta <0:
     print("Pas de solutions")
elif delta ==0:
    x1 = -b / (2*a)
    print("Une seule solution :",x)
else:
    x1 = (-b - sqrt(delta) / (2*a))
    x2 = (-b + sqrt(delta) / (2*a))
   print("Deux solutions :",x1,x2)
```

- Créer une fonction *resol_eq_second_degre (a,b,c)* prenant en paramètres les coefficients de l'équation du second degré et retournant le nombre de solutions ainsi que leur valeur.

```
def resol_eq_second_degre (a,b,c) :
   a = float(input("Entrez a : "))
   b = float(input("Entrez b : "))
   c = float(input("Entrez c : "))
   delta = b**2 - 4*a*c
if delta <0:
   return 0 , "aucune"
```

```
elif delta ==0:
   x = -b / (2*a)
   return 1 , x

else:
   x1 = (-b - sqrt(delta) / (2*a))
   x2 = (-b + sqrt(delta) / (2*a))
   return 2 ,x1,x2
```